

418 Project Proposal

garias, johnnyt

March 2026

1 Title

Ray Tracing with a Parallel BVH tree. By Gabriel Arias and Johnny Tran.

2 URL

https://garias1831.github.io/15418_project_site/

3 Summary

We will implement ray tracing with a Bounding Volume Hierarchy (BVH) tree on NVIDIA GPUs in the lab. We will focus on rendering a dynamic scene with a BVH as an additional optimization, which allows us to group nearby objects in the scene together and reduce ray-tracing cost, instead of having to run the ray-tracing algorithm per object in the scene.

4 Background

This project is a combination of two parallelization tasks. One task is the ray tracing aspect, which is embarrassingly parallel. Ray tracing is a computer graphics rendering technique that is used to simulate the physical movement of light rays to create more realistic images. These rays are simulated to bounce for some number of times off of different objects to see how the a pixel should be colored. This task is embarrassingly parallel, because ray tracing sends out rays for each pixel, making the task simple to parallelize as threads can be assigned to each pixel either statically or dynamically. Since pixels are independent of each other and do not need to share information, there is nothing particularly interesting to parallelize.

The other task we will parallelize will be the more interesting task. While naively detecting what objects a ray hits in raytracing is expensive due to having to check for collisions per object in a scene, Bounding Volume Hierachies (BHV's) helps reduce this computational cost significantly. BVHs solve the issue of checking all of these collisions by placing all of the low detail objects such as

triangles into a tree. Within a tree, these triangles are placed at the leaves and its parent nodes represent a bounding box containing all of the objects that its children represents. So, using this structure, rays can instead check against these bounding boxes instead of checking against every object which skips a lot of collision checks.

BVH trees are typically built by recursively partitioning a set of geometric primitives into smaller subsets. Starting with all objects in a root node, the algorithm chooses a split along some axis based on a heuristic such as the Surface Area Heuristic (SAH) or a simpler median split. The primitives are then divided into two groups, each enclosed in a bounding box, forming the left and right child nodes. This process continues recursively until a stopping condition is met, such as reaching a small number of primitives per leaf.

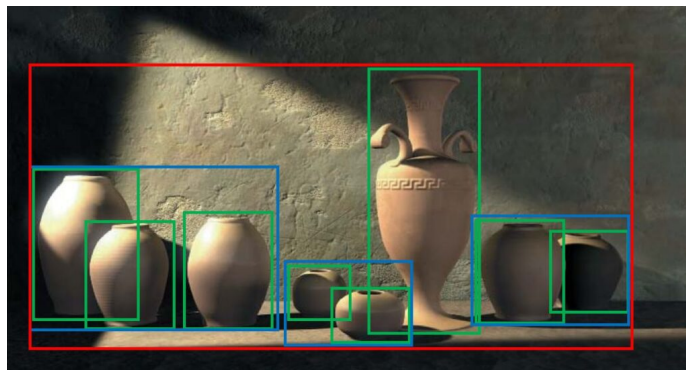


Figure 1: Credit: <https://jacco.ompf2.com/2022/04/13/how-to-build-a-bvh-part-1-basics/>

5 Challenge

The most interesting part of the project will revolve around how we construct and maintain the BVH tree. While standard ray-tracing is easy to parallelize (as we only have to compute ray intersections independently per-pixel), the BVH tree optimization may prove significantly harder to scale. In a dynamic scene, the objects are allowed to move around freely; this means that they can leave their assigned bounding volumes during the scene. This means that we will have to figure out a way to maintain the correctness of the tree while minimizing computational costs. This presents an obvious load balance concern, as the number of objects a processor may change and become uneven over time. This may also be a place where we need to communicate, as a drifting set of objects may change the structure of the shared BVH tree (e.g consider inner volume A contained in B drifting out of B). Locality is another concern; while we want to minimize the number of bounding volumes to reduce intersect checks

+ BVH queries, we also need to ensure that the assigned volumes are sufficiently "tight" around the objects that they cover to improve render quality.

With regards to the specifics of the workload, BVH trees introduce hierarchical dependencies where for initialization and recomputation, parent nodes must be computed before their children nodes can be computed, following a top-down dependency. On the other hand, for reconstruction when only some objects move out of certain boundaries, we will likely need to implement something with a bottom-up approach to move objects into the correct bounding boxes. We also expect our memory accesses to be more irregular since our objects may occasionally be close to each other in memory, but will become worse as the BVH tree splits more since objects will be read in a unpredictable order.

For divergence during BVH traversal, where different rays follow different paths through the tree and cause threads within a warp to execute different branches which reduces the efficiency of computation on GPUs. Additionally, BVH construction has a higher communication to computation ratio compared to ray tracing, as it requires coordination between threads for partitioning objects and updating shared data structures.

6 Resources

Due to the main focus of the project being on the construction and parallelization of the BVH tree, we plan to reference existing ray-tracing algorithms / implementations to complement our own. We will likely need to make use of starter code to implement some of the overhead of rendering the scenes, as well as libraries for efficient math operations (mostly linear algebra). We plan to implement the BVH code and parallelization strategy from scratch.

Some preliminary sources:

- https://en.wikipedia.org/wiki/Bounding_volume_hierarchy
- <https://vchizhov.github.io/resources/ray%20tracing/ray%20tracing%20tutorial%20series%20vchizhov>
- <https://geometrian.com/resources/tutorials/rtaccel>

In addition, while we don't expect to need more GPU compute, it could be interesting to see what speedup on the tree rebuilds we obtain with more compute.

7 Goals and Deliverables

Planned Goals

We plan to have a correct parallel ray tracer that uses the above BVH tree to efficiently render scenes. The above algorithm should be able to handle moving / dynamic scenes efficiently. On a standard scene, we want the BVH implementation to have good scaling with good render quality. We specifically

want to emphasize scalability as the core ray-tracing algorithm should have high parallelism, and we want the BVH tree to hinder that parallelism as little as possible. In the case that work goes more slowly, we may sacrifice these goals for the sake of correctness.

Stretch Goals

It might be interesting to compare performance of the different iterations of the algorithm: sequential ray-tracing, parallel ray-tracing, sequential BVH and parallel BVH. If we have time at the end of the project, benchmarking all these could be an interesting way to compare the optimizations. Another interesting enhancement could be operating on full 3D objects (e.g spheres), which may involve leveraging a graphics library like OpenGL. This may also require a non-trivial rewrite of the existing code, which may prove challenging to implement in the core stage of the project.

8 Platform Choice

We will use C++ and CUDA because we plan on using the NVIDIA lab GPUs, which are compatible with C/C++. We choose C++ over C due to the fact that C++ makes using data structures a bit more convenient.

9 Schedule

Week 1 (3/25-4/1): As this is an exam week, the main task is read up and fully understand the implementation. If possible, get very basic starter code for just the ray tracing.

Week 2 (4/1-4/8): Implement a sequential version of the ray tracing and BVH tree.

Week 3: (4/8-4/15): Parallelize the BVH tree on a static and dynamic scene

Week 4: (4/15-4/22): Debug and benchmark BVH tree on different scenes. Create graphs from these benchmarks

Week 5: (4/22-4/30): Finish poster, website, write up

Week 6: (5/1): Present project